

Aufgabenstellung: Interfaces in Java

(Interfaces „Druckbar“, „PDFDruckbar“, „Mailbar“, „HatFläche“ sowie Klassen „Dokument“, „PDFDokument“, „Kreis“, „Rechteck“ und Arbeiten mit implementierenden Klassen, Interface-Vererbung und polymorpher Nutzung)

Ziel der Aufgabe

In dieser Aufgabe analysieren und entwickeln Sie ein Java-Programm, das mehrere Interfaces und dazugehörige konkrete Klassen verwendet.

Sie lernen dabei,

- was ein Interface in Java ist,
- wie Klassen Interfaces implementieren,
- wie ein Interface ein anderes Interface erweitern kann,
- wie mehrere Interfaces gleichzeitig verwendet werden,
- wie Interfaces als Variablen-, Listen- und Parametertyp eingesetzt werden,
- wie gemeinsame Fähigkeiten unabhängig von konkreten Klassen beschrieben werden.

Der Fokus liegt auf dem Verständnis, dass ein Interface in Java einen

Vertrag beschreibt, den Klassen erfüllen müssen. Klassen können dadurch gemeinsame Fähigkeiten besitzen, auch wenn sie inhaltlich unterschiedlich aufgebaut sind.

Fachlicher Hintergrund

In der objektorientierten Programmierung gibt es Situationen, in denen nicht eine gemeinsame Oberklasse im Mittelpunkt steht, sondern eine gemeinsame Fähigkeit.

Ein Objekt kann zum Beispiel:

- gedruckt werden,
- als PDF gedruckt werden,
- per Mail versendet werden,
- eine Fläche berechnen.

Solche Fähigkeiten lassen sich in Java mit Interfaces beschreiben.

Ein Interface kann:

- Methodensignaturen vorgeben,
- Konstanten enthalten,
- default-Methoden enthalten,
- statische Methoden enthalten.

Typische Beziehungen in dieser Aufgabe sind:

- ein Dokument ist druckbar,
- ein PDF-Dokument ist druckbar,
- ein PDF-Dokument ist zusätzlich mailbar,
- ein Rechteck hat eine Fläche,
- ein Kreis hat eine Fläche.

Damit eignet sich dieses Beispiel gut, um den Unterschied zwischen konkreten Klassen und allgemeinen Fähigkeiten zu verdeutlichen.

Vorgegebene Klassen- und Interfacestruktur

Im Package `template` beziehungsweise in den Unterpackages sollen folgende Interfaces und Klassen entwickelt und verwendet werden:

- `Druckbar`
- `PDFDruckbar`
- `Mailbar`
- `Dokument`
- `PDFDokument`
- `HatFläche`

- Kreis
- Rechteck
- Main

Die Bestandteile haben dabei folgende Rollen:

- Druckbar ist ein grundlegendes Interface für druckbare Objekte,
- PDFDruckbar erweitert Druckbar,
- Mailbar beschreibt die Fähigkeit, per Mail versendet zu werden,
- Dokument ist eine konkrete Klasse, die Druckbar implementiert,
- PDFDokument ist eine konkrete Klasse, die von Dokument erbt und zusätzlich PDFDruckbar und Mailbar implementiert,
- HatFläche ist ein Interface für Objekte mit Flächenberechnung,
- Kreis und Rechteck sind konkrete Klassen, die HatFläche implementieren,
- Main enthält die main-Methode zum Testen des Programms.

Aufgabe 1: Entwicklung des Interfaces „Druckbar“

Ziel

Sie entwickeln das Interface Druckbar als allgemeine Beschreibung für alle Objekte, die gedruckt werden können.

Zu entwickelnde Struktur

Das Interface Druckbar soll mindestens enthalten:

```
public static final String DIN_A4
```

sowie passende Methoden, zum Beispiel:

```
public abstract void drucken()
```

Zusätzlich soll das Interface weitere Möglichkeiten von Interfaces zeigen, zum Beispiel:

```
public static void testDruck()  
public default boolean prüfeFormat(String format)
```

Anforderungen

Funktionale Anforderungen

Entwickeln Sie das Interface Druckbar so, dass:

- eine gemeinsame Druckfähigkeit beschrieben wird,
- eine Konstante für ein Seitenformat vorhanden ist,
- eine Methode zum Drucken vorgegeben wird,
- eine statische Methode für einen Testdruck aufgerufen werden kann,
- eine default-Methode zur Formatprüfung vorhanden ist.

Technische Anforderungen

Verwenden Sie:

- das Schlüsselwort `interface`,
- eine Konstante `DINA4`,
- mindestens eine abstrakte Methode,
- mindestens eine statische Methode,
- mindestens eine `default`-Methode.

Ziel dieser Aufgabe

Sie sollen verstehen,

- was ein Interface in Java ist,
- welche Bestandteile ein Interface enthalten kann,
- dass ein Interface selbst keine konkrete Objekterzeugung beschreibt,
- wie gemeinsame Fähigkeiten allgemein festgelegt werden.

Aufgabe 2: Entwicklung der Klasse „Dokument“

Ziel

Sie entwickeln die Klasse `Dokument` als konkrete Klasse, die das Interface `Druckbar` implementiert.

Zu entwickelnde Klassenstruktur

```
public class Dokument implements Druckbar
```

Anforderungen

Funktionale Anforderungen

Entwickeln Sie die Klasse Dokument so, dass:

- sie einen Text speichern kann,
- der Text über den Konstruktor gesetzt wird,
- die Druckmethode des Interfaces konkret umgesetzt wird,
- der Text über Getter- und gegebenenfalls Setter-Methoden zugänglich ist.

Technische Anforderungen

Verwenden Sie:

- `implements Druckbar`,
- ein Attribut `text`,
- einen Konstruktor zur Initialisierung,
- mindestens eine Getter-Methode,
- eine konkrete Implementierung der Methode `drucken()`.

Ziel dieser Aufgabe

Sie sollen verstehen,

- wie eine Klasse ein Interface implementiert,
 - dass die im Interface vorgegebenen Methoden konkret umgesetzt werden müssen,
 - wie aus einer allgemeinen Fähigkeit eine konkrete Klassenfunktion wird.
-

Aufgabe 3: Entwicklung des Interfaces „PDFDruckbar“

Ziel

Sie entwickeln das Interface PDFDruckbar als Spezialisierung von Druckbar.

Zu entwickelnde Struktur

```
public interface PDFDruckbar extends Druckbar
```

Anforderungen

Funktionale Anforderungen

Entwickeln Sie das Interface PDFDruckbar so, dass:

- es alle Merkmale von Druckbar übernimmt,
- zusätzlich eine Methode zum PDF-Drucken vorgibt.

Technische Anforderungen

Verwenden Sie:

- `extends Druckbar`,
- mindestens eine zusätzliche Methodendeklaration, zum Beispiel `pdfDrucken()`.

Ziel dieser Aufgabe

Sie sollen verstehen,

- dass Interfaces andere Interfaces erweitern können,
 - dass ein spezialisiertes Interface die Merkmale eines allgemeineren Interfaces übernimmt,
 - wie sich Fähigkeiten in Java schrittweise spezialisieren lassen.
-

Aufgabe 4: Entwicklung des Interfaces „Mailbar“

Ziel

Sie entwickeln das Interface `Mailbar` als allgemeine Beschreibung für Objekte, die per Mail versendet werden können.

Anforderungen

Funktionale Anforderungen

Entwickeln Sie das Interface `Mailbar` so, dass:

- eine Methode zum Versenden per Mail vorgegeben wird.

Technische Anforderungen

Verwenden Sie:

- `interface`,
- mindestens eine Methodendeklaration, zum Beispiel `mailVersenden()`.

Ziel dieser Aufgabe

Sie sollen verstehen,

- dass verschiedene Fähigkeiten in getrennten Interfaces beschrieben werden können,
- wie ein Objekt später mehrere Fähigkeiten gleichzeitig besitzen kann.

Aufgabe 5: Entwicklung der Klasse „PDFDokument“

Ziel

Sie entwickeln die Klasse `PDFDokument` als konkrete Spezialisierung von `Dokument`.

Zu entwickelnde Klassenstruktur

```
public class PDFDokument extends Dokument implements  
PDFDruckbar, Mailbar
```

Anforderungen

Funktionale Anforderungen

Entwickeln Sie die Klasse `PDFDokument` so, dass:

- sie den Text eines Dokuments übernehmen kann,
- sie zusätzlich als PDF gedruckt werden kann,
- sie zusätzlich per Mail versendet werden kann,
- sie die bereits vorhandene Druckfunktion von `Dokument` nutzen kann.

Technische Anforderungen

Verwenden Sie:

- `extends Dokument`,
- `implements PDFDruckbar, Mailbar`,
- einen Konstruktor mit `super(...)`,
- eine konkrete Implementierung von `pdfDrucken()`,
- eine konkrete Implementierung von `mailVersenden()`.

Ziel dieser Aufgabe

Sie sollen verstehen,

- wie Klassenvererbung und Interface-Implementierung kombiniert werden,
 - dass eine Klasse mehrere Interfaces gleichzeitig umsetzen kann,
 - wie vorhandene Funktionalität wiederverwendet und erweitert werden kann.
-

Aufgabe 6: Entwicklung des Interfaces „HatFläche“

Ziel

Sie entwickeln das Interface `HatFläche` als allgemeine Beschreibung für Objekte, deren Fläche berechnet werden kann.

Zu entwickelnde Struktur

```
public interface HatFläche
```

mit einer Methode wie:

```
double berechneFläche()
```

Anforderungen

Funktionale Anforderungen

Entwickeln Sie das Interface `HatFläche` so, dass:

- eine gemeinsame Fähigkeit zur Flächenberechnung beschrieben wird.

Technische Anforderungen

Verwenden Sie:

- `interface`,
- mindestens eine Methodendeklaration zur Flächenberechnung.

Ziel dieser Aufgabe

Sie sollen verstehen,

- dass Interfaces nicht nur für Dokumente oder Ausgabefunktionen, sondern auch für fachliche Modelle genutzt werden können,
- wie eine gemeinsame Fähigkeit unabhängig von der konkreten Form beschrieben wird.

Aufgabe 7: Entwicklung der Klassen „Kreis“ und „Rechteck“

Ziel

Sie entwickeln zwei konkrete Klassen, die das Interface `HatFläche` implementieren.

Zu entwickelnde Klassenstruktur

```
public class Kreis implements HatFläche
public class Rechteck implements HatFläche
```

Anforderungen

Funktionale Anforderungen

Entwickeln Sie die Klassen so, dass:

- Kreis einen Radius besitzt,
- Rechteck zwei Seitenlängen besitzt,
- beide Klassen die Methode `berechneFläche()` jeweils passend umsetzen,
- die Flächenwerte korrekt berechnet werden.

Technische Anforderungen

Verwenden Sie:

- `implements HatFläche`,
- passende Attribute zur Speicherung der Maße,
- Konstruktoren zur Initialisierung,
- Getter- und gegebenenfalls Setter-Methoden,
- eine jeweils eigene Implementierung von `berechneFläche()`.

Ziel dieser Aufgabe

Sie sollen verstehen,

- dass unterschiedliche Klassen dieselbe Fähigkeit auf

unterschiedliche Weise realisieren können,

- wie ein Interface gemeinsame Struktur vorgibt, ohne die konkrete Berechnung festzulegen.

Aufgabe 8: Analyse von Interface-Methoden

Ziel

Sie analysieren die verschiedenen Arten von Methoden in Interfaces.

Anforderungen

Funktionale Anforderungen

Untersuchen und erläutern Sie die Klassen und Interfaces so, dass deutlich wird:

- welche Methoden abstrakt vorgegeben werden,
- welche Methode als default-Methode bereits eine Implementierung besitzt,
- welche Methode als statische Methode direkt über das Interface aufgerufen wird,
- wie diese unterschiedlichen Methodenarten im Programm verwendet werden.

Technische Anforderungen

Beziehen Sie sich auf:

- `drucken()`,
- `prüfeFormat(...)`,
- `testDruck()`.

Ziel dieser Aufgabe

Sie sollen verstehen,

- worin sich abstrakte, default- und statische Methoden in Interfaces unterscheiden,
- wie diese Methodentypen praktisch eingesetzt werden.

Aufgabe 9: Entwicklung der Testklasse „Main“

Ziel

Sie entwickeln ein Testprogramm, das die Verwendung der Interfaces und Klassen praktisch demonstriert.

Zu entwickelnde Klasse

Die Klasse `Main` soll eine `main`-Methode enthalten.

Anforderungen

Funktionale Anforderungen

Das Testprogramm soll:

1. die statische Methode des Interfaces `Druckbar` aufrufen,
2. ein Objekt der Klasse `Dokument` erzeugen,
3. die `default`-Methode zur Formatprüfung verwenden,
4. ein Objekt von `PDFDokument` über den Typ `PDFDruckbar` ansprechen,
5. ein Objekt von `PDFDokument` über den Typ `Mailbar` ansprechen,
6. eine Liste vom Typ `List<Druckbar>` verwenden,
7. mehrere druckbare Objekte in die Liste einfügen,
8. über die Liste alle Objekte drucken,
9. ein Rechteck und einen Kreis erzeugen,
10. eine Methode verwenden, die als Parameter den Interface-Typ `HatFläche` erhält.

Technische Anforderungen

Verwenden Sie:

- eine `main`-Methode,
- mehrere Objekterzeugungen,
- Methodenaufrufe auf Objekten mit verschiedenen Interface-Typen,
- eine `List<Druckbar>`,

- eine Methode wie `ausgabeFläche(HatFläche hatFläche)`,
- sinnvolle Konsolenausgaben.

Ziel dieser Aufgabe

Sie sollen verstehen,

- wie Interfaces in einem Testprogramm praktisch eingesetzt werden,
- wie ein Objekt über verschiedene Typen betrachtet werden kann,
- wie polymorphe Programmierung mit Interfaces funktioniert.

—

Aufgabe 10: Analyse der Typbeziehungen

Ziel

Sie analysieren die Beziehungen zwischen Interfaces, Klassen und Objektreferenzen.

Anforderungen

Funktionale Anforderungen

Erklären Sie,

- warum von einem Interface kein direktes Objekt erzeugt werden kann,
- warum `Dokument` als `Druckbar` betrachtet werden kann,

- warum PDFDokument gleichzeitig Druckbar, PDFDruckbar und Mailbar sein kann,
- warum Kreis und Rechteck beide an eine Methode mit dem Parametertyp HatFläche übergeben werden können,
- welchen Vorteil es hat, mit Interface-Typen statt mit konkreten Klassentypen zu arbeiten.

Technische Anforderungen

Beziehen Sie sich auf:

- implements,
- extends,
- Interface als Variablentyp,
- Interface als Listentyp,
- Interface als Methodentyp.

Ziel dieser Aufgabe

Sie sollen verstehen,

- wie Interfaces Klassenbeziehungen strukturieren,
- wie gemeinsame Fähigkeiten statt gemeinsamer Herkunft modelliert werden,
- wie Programme dadurch flexibler und erweiterbarer werden.

Lernziele

Nach Abschluss dieser Aufgabe können Sie:

- erklären, was ein Interface in Java ist,
 - den Unterschied zwischen Interface und konkreter Klasse beschreiben,
 - Klassen mit `implements` modellieren,
 - Interface-Vererbung mit `extends` anwenden,
 - abstrakte, `default`- und statische Methoden in Interfaces unterscheiden,
 - mehrere Interfaces in einer Klasse kombinieren,
 - Interfaces als Variablen-, Listen- und Parametertyp verwenden,
 - die Struktur in einem Testprogramm sichtbar machen.
-

Didaktische Einordnung

Diese Aufgabe erweitert grundlegende Java-Kenntnisse um das Konzept der Interfaces.

Sie knüpft an folgende Themen an:

- Klassen und Objekte
- Attribute und Methoden

- Konstruktoren
- Vererbung
- `extends`
- `implements`
- Interfaces
- `default`-Methoden
- statische Methoden in Interfaces
- Polymorphie
- objektorientierte Modellierung

Die Aufgabe zeigt, dass Interfaces in Java genutzt werden, um gemeinsame Fähigkeiten unabhängig von einer konkreten Klassenhierarchie zu beschreiben.

Sie bildet eine wichtige Grundlage für weiterführende Themen wie:

- Mehrfachimplementierung
- Polymorphie
- dynamische Bindung
- abstrakte Klassen
- UML-Klassendiagramme
- objektorientierter Softwareentwurf

Reflexions- und Kontrollfragen

Kontrollfragen zu Interfaces

1. Was ist ein Interface in Java?
2. Warum kann von einem Interface kein direktes Objekt erzeugt werden?
3. Welche Aufgabe hat ein Interface?
4. Worin unterscheidet sich ein Interface von einer konkreten Klasse?

Kontrollfragen zur Klassen- und Interface-Struktur

5. Warum implementiert `Dokument` das Interface `Druckbar`?
6. Warum erweitert `PDFDruckbar` das Interface `Druckbar`?
7. Warum kann `PDFDokument` mehrere Interfaces gleichzeitig implementieren?
8. Warum implementieren `Kreis` und `Rechteck` beide `HatFläche`?

Kontrollfragen zur Programmlogik

9. Was ist der Unterschied zwischen `implements` und `extends`?
10. Was ist eine `default`-Methode?
11. Was ist eine statische Methode in einem Interface?
12. Warum ist es sinnvoll, Interfaces als Parameter- oder Listentyp zu

verwenden?

13. Warum kann eine Methode mit dem Parametertyp `HatFläche` sowohl einen Kreis als auch ein Rechteck verarbeiten?
14. Welche Vorteile bietet diese Modellierung für Erweiterbarkeit und Wiederverwendung?
